

Introduction to Cryptography

Musard Balliu

Lecture 3

DD2395/DD2391

Last Lecture

- Introduction to cryptography
- Security properties of interest
- Symmetric key cryptography
 - One-Time Pad

Questions?

Today's Plan

- **Symmetric key cryptography**
 - Block cyphers, Hashes, MACs, Authenticated encryption
- **Asymmetric key cryptography**
 - Problem definition, digital signatures

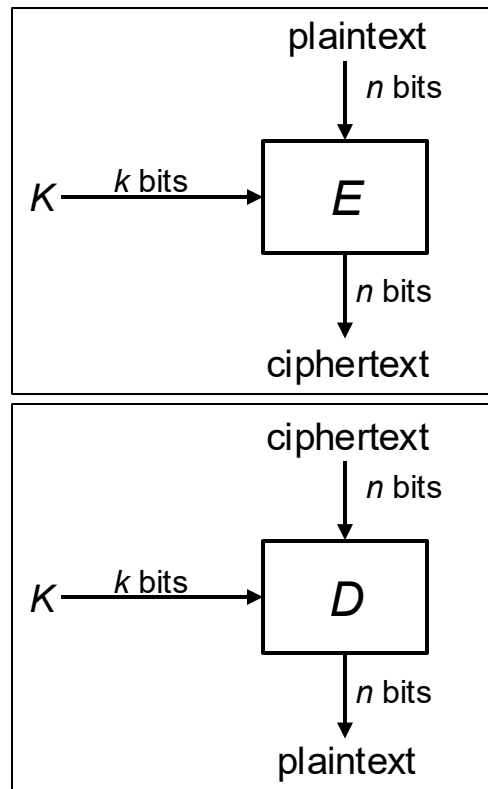
Block Ciphers

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	● One-time pads ● Block ciphers (e.g. AES)	● RSA encryption ● ElGamal encryption
Integrity, Authentication	● Hash functions ● MACs (e.g. HMAC)	● Digital signatures (e.g. RSA signatures)

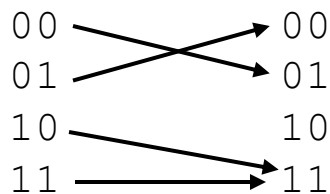
Block Ciphers: Definition

- **Block cipher:** An encryption/decryption algorithm that encrypts a fixed-sized block of bits
- $E(K, M) \rightarrow C$: Encryption
 - Inputs: k -bit key K and an n -bit plaintext M
 - Output: An n -bit ciphertext C
 - Sometimes written as: $\{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$
- $D(K, C) \rightarrow M$: Decryption
 - Inputs: a k -bit key, and an n -bit ciphertext C
 - Output: An n -bit plaintext
 - Sometimes written as: $\{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$
 - The inverse of the encryption function
- Properties
 - **Correctness:** E_K is a permutation, D_K is its inverse
 - **Efficiency:** Encryption/decryption should be fast
 - **Security:** E behaves like a random permutation

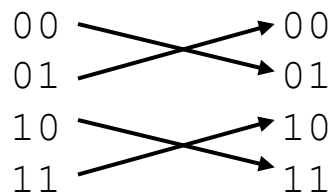


Block Ciphers: Correctness

- $E(K,M)$ must be a **permutation (bijective function)** on n -bit strings
 - Each input must correspond to exactly one unique output
- Intuition
 - Suppose $E(K,M)$ is not bijective
 - Then two inputs might correspond to the same output: $E(K, x_1) = E(K, x_2) = y$
 - Given ciphertext y , you can't uniquely decrypt. $D(K, y) = x_1?$ $D(K, y) = x_2?$



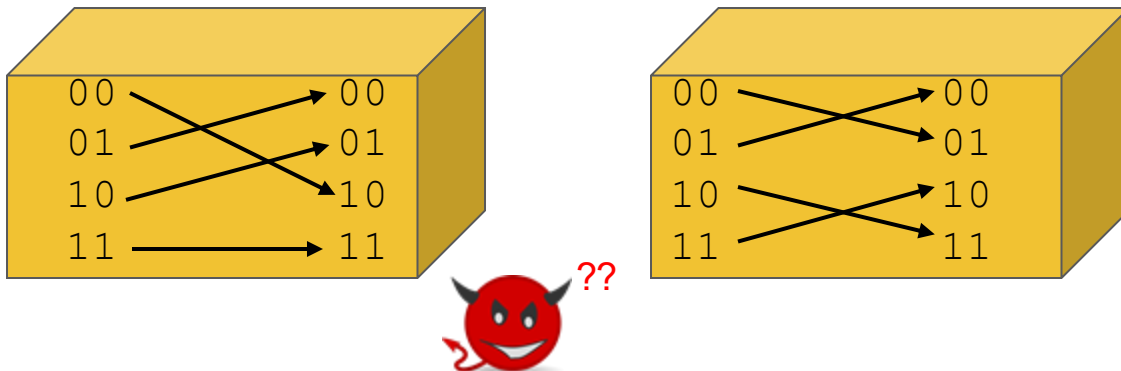
Not bijective: Two inputs encrypt to the same output



Bijjective: Each input maps to exactly one unique output

Block Ciphers: Security

- A secure block cipher behaves like a randomly chosen permutation from the set of all permutations on n -bit strings
 - A random permutation: Each n -bit input is mapped to one randomly-chosen n -bit output
- Defined by a distinguishing game
 - Eve gets two boxes: One is a randomly chosen permutation, and one is E_K with a randomly chosen key K
 - Eve should not be able to tell which is which with probability $> 1/2$



One of these is E_K with a randomly chosen K , and the other one is a randomly chosen permutation. Eve can't distinguish them.

Block ciphers: Brute-force attacks?

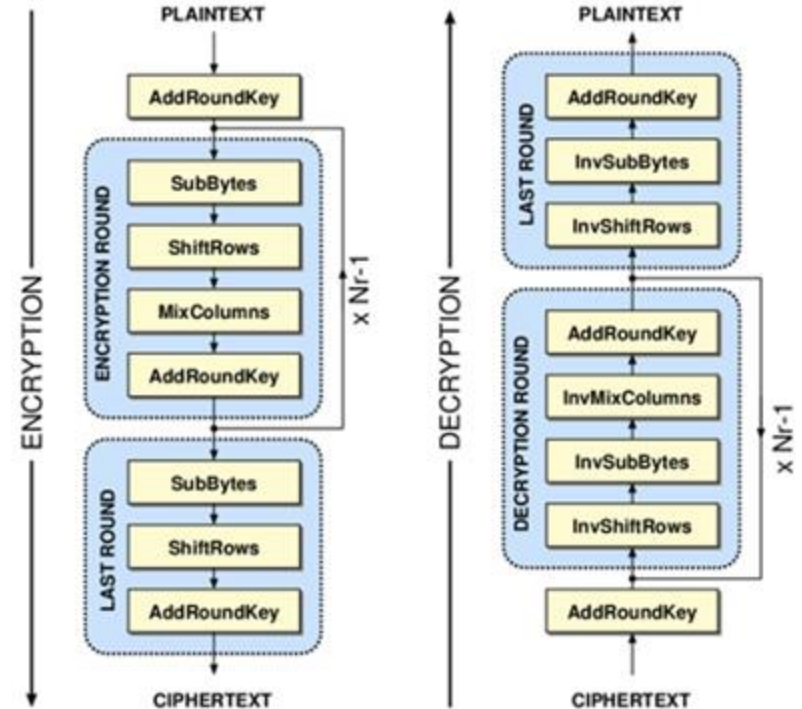
- How hard is it to run a brute-force attack on a 128-bit key?
 - We have to try 2^{128} possibilities. How big is 2^{128} ?
- Handy approximation: $2^{10} \approx 10^3$
 - $2^{128} = 2^{10 \cdot 12.8} \approx (10^3)^{12.8} \approx (10^3)^{13} = 10^{39}$
- Suppose we have massive hardware that can try 10^9 (1 billion) keys in 1 nanosecond (a billionth of a second). That's 10^{18} keys per second
 - We'll need $10^{39} / 10^{18} = 10^{21}$ seconds. How long is that?
 - One year $\approx 3 \times 10^7$ seconds
 - 10^{21} seconds / $3 \times 10^7 \approx 3 \times 10^{13}$ years ≈ 30 trillion years
- **Takeaway:** Brute-forcing a 128-bit key takes astronomically long. Don't even try.

AES (Advanced Encryption Standard)

- Key size 128, 192, or 256 bits ($k = 128, 192, \text{ or } 256$)
 - Use key size 256 these days
- Block size 128 bits ($n = 128$)
 - Note: The block size is still always 128 bits, regardless of key size
- You don't need to know how AES works, but you do need to know its parameters

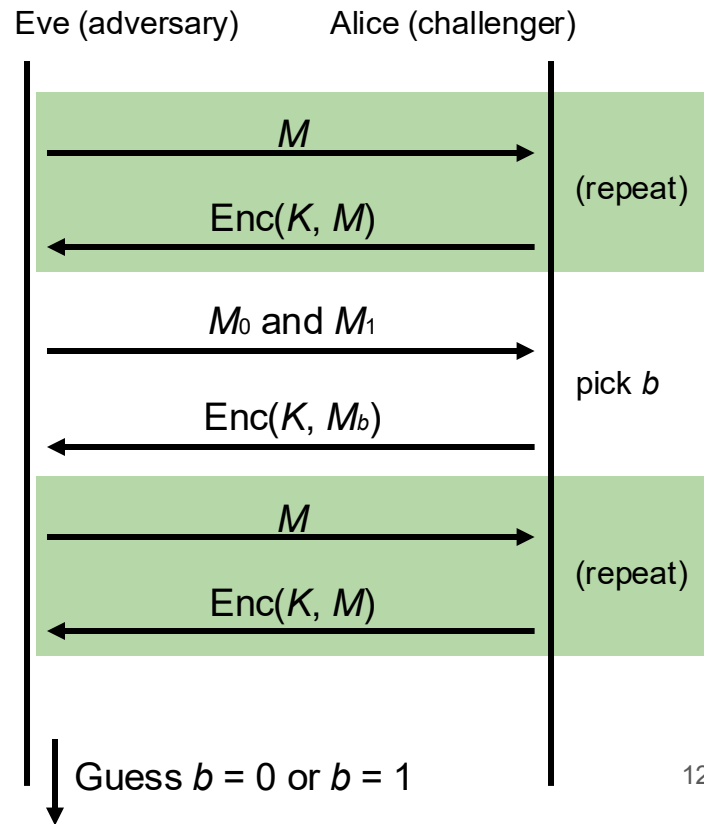
AES Algorithm

- Take the plaintext and shuffle it using the key
- Different key sizes use different numbers of rounds
 - 10 rounds for 128-bit keys
 - 12 rounds for 192-bit keys
 - 14 rounds for 256-bit keys
- Each round uses its own “round key” derived from the cipher key
- Each round:
 - SubBytes()
 - ShiftRows()
 - MixColumns() (if not last round)
 - AddRoundKey()



Are Block Ciphers IND-CPA Secure?

- Consider the following adversary:
 - Eve sends two different messages M_0 and M_1
 - Eve receives either $Enc(K, M_0)$ or $Enc(K, M_1)$
 - Eve requests the encryption of M_0 again
 - Strategy: If the encryption of M_0 matches what she received, guess $b = 0$. Else, guess $b = 1$.
- Eve can win the IND-CPA game with probability 1!
 - Block ciphers are not IND-CPA secure



Issues with Block Ciphers

- Block ciphers are not IND-CPA secure, because they're deterministic
 - A scheme is **deterministic** if the same input always produces the same output
 - No deterministic scheme can be IND-CPA secure because the adversary can always tell if the same message was encrypted twice
- Block ciphers can only encrypt messages of a fixed size
 - For example, AES can only encrypt-decrypt 128-bit messages
 - What if we want to encrypt something longer than 128 bits?
- To address these problems, we'll add **modes of operation** that use block ciphers as a building block – stay tuned until Lecture 5

Summary: Block Ciphers

- Encryption: input a k -bit key and n -bit plaintext, receive n -bit ciphertext
- Decryption: input a k -bit key and n -bit ciphertext, receive n -bit plaintext
- Correctness: when the key is fixed, $E(K,M)$ should be bijective
- Security
 - Without the key, $E(K,M)$ is computationally indistinguishable from a random permutation
 - Brute-force attacks take astronomically long and are not possible
- Efficiency: algorithms use XORs and bit-shifting (very fast)
- Implementation: AES is the modern standard
- Issues
 - Not IND-CPA secure because they're deterministic
 - Can only encrypt n -bit messages

Cryptographic Hashes

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	● One-time pads ● Block ciphers (e.g. AES)	● RSA encryption ● ElGamal encryption
Integrity, Authentication	● Hash functions ● MACs (e.g. HMAC)	● Digital signatures (e.g. RSA signatures)

Cryptographic Hash Function: Definition

- Hash function: $H(M)$
 - Input: *Arbitrary* length message M
 - Output: *Fixed* length, n -bit hash
 - Sometimes written as $\{0, 1\}^* \rightarrow \{0, 1\}^n$
- Properties
 - **Correctness:** Deterministic
 - Hashing the same input always produces the same output
 - **Efficiency:** Efficient to compute
 - **Security:** One-way-ness (“preimage resistance”)
 - **Security:** Collision-resistance (weak and strong)
 - **Security:** Random/unpredictability, no predictable patterns for how changing the input affects the output
 - Changing 1 bit in the input causes the output to be completely different
 - Also called “random oracle” assumption

Hash Function: Intuition

- A hash function provides a fixed-length “fingerprint” over a sequence of bits
- Example: Document comparison
 - If Alice and Bob both have a 1 GB document, they can both compute a hash over the document and (securely) communicate the hashes to each other
 - If the hashes are the same, the files must be the same, since they have the same “fingerprint”
 - If the hashes are different, the files must be different

Hash Function: One-way-ness or Preimage Resistance

- **Informal:** Given an output y , it is infeasible to find *any* input x such that $H(x) = y$
- **More formally:** For all polynomial time adversary,
Pr[x chosen randomly from plaintext space; $y = H(x)$:
Adv(y) outputs x' s.t. $H(x') = y$] is negligible
- Intuition: Here's an output. Can you find an input that hashes to this output?
 - Note: The adversary just needs to find *any* input, not necessarily the input that was actually used to generate the hash
- Example: Is $H(x) = 1$ one-way?
 - No, because given output 1, an attacker can return any number x

Hash Function: (Strong) Collision Resistance

- **Collision:** Two different inputs with the same output
 - $x \neq x'$ and $H(x) = H(x')$
 - Can we design a hash function with no collisions?
 - No, because there are more inputs than outputs (pigeonhole principle)
 - However, we want to make finding collisions *infeasible* for an attacker
- **Collision resistance:** It is infeasible to (i.e. no polynomial time attacker can) find any pair of inputs $x' \neq x$ such that $H(x) = H(x')$
- Intuition: Can you find *any* two inputs that collide with the same hash output for *any* output?
- Also referred to as **strong collision resistance**

Hash Function: Second Preimage Resistance

- **Collision:** Two different inputs with the same output
 - $x \neq x'$ and $H(x) = H(x')$
 - Can we design a hash function with no collisions?
 - No, because there are more inputs than outputs (pigeonhole principle)
 - However, we want to make finding collisions *infeasible* for an attacker
- **Second preimage resistance:** Given an input x , it is infeasible to (i.e. no polynomial time attacker can) find another input x' such that $x' \neq x$ but $H(x) = H(x')$
- Intuition: Given an input x , can you find *any* input that collides with the same hash output $H(x)$?
- Note that strong collision resistance implies second preimage resistance (also called **weak collision resistance**). Why?
- [Homework exercise 1](#): Implement a function to find collisions

Hash Function: Examples

- MD5
 - Output: 128 bits
 - Security: Completely broken
- SHA-1
 - Output: 160 bits
 - Security: Completely broken in 2017
 - Was known to be weak before 2017, but still used sometimes
- SHA-2
 - Output: 256, 384, or 512 bits (sometimes labeled SHA-256, SHA-384, SHA-512)
 - Not currently broken, but some variants are vulnerable to a length extension attack
 - Current standard
- SHA-3
 - Output: 256, 384, or 512 bits
 - Current standard (not meant to replace SHA-2, just a different construction)

Length Extension Attacks

- **Length extension attack:** Given $H(x)$ and the length of x , but not x , an attacker can create $H(x || m)$ for any m of the attacker's choosing
 - Note: This doesn't violate any property of hash functions but is undesirable in some circumstances
- SHA-256 (256-bit version of SHA-2) is vulnerable
- SHA-3 is not vulnerable

Do hashes provide integrity?

- It depends on your threat model
- Scenario
 - Mozilla publishes a new version of Firefox on some download servers
 - Alice downloads the program binary
 - How can she be sure that nobody tampered with the program?
- Idea: use cryptographic hashes
 - Mozilla hashes the program binary and publishes the hash on its website
 - Alice hashes the binary she downloaded and checks that it matches the hash on the website
 - If Alice downloaded a malicious program, the hash would not match (tampering detected!)
 - An attacker can't create a malicious program with the same hash (collision resistance)
- Threat model: We assume the attacker cannot modify the hash on the website
 - We have integrity, as long as we can communicate the hash securely

Do hashes provide integrity?

- It depends on your threat model
- Scenario
 - Alice and Bob want to communicate over an insecure channel
 - Mallory might tamper with messages
- Idea: Use cryptographic hashes
 - Alice sends her message with a cryptographic hash over the channel
 - Bob receives the message and computes a hash on the message
 - Bob checks that the hash he computed matches the hash sent by Alice
- Threat model: Mallory can modify the message *and the hash*
 - No integrity!

Do hashes provide integrity?

- It depends on your threat model
- If the attacker can modify the hash, hashes don't provide integrity
- Main issue: Hashes are *unkeyed* functions
 - There is no secret key being used as input, so any attacker can compute a hash on any value
- Next: Use hashes to design schemes that provide integrity

How to Provide Integrity

- Reminder: We're still in the symmetric-key setting
 - Assume that Alice and Bob share a secret key, and attackers don't know the key
- We want to attach some piece of information to *prove* that someone with the key sent this message
 - This piece of information can only be generated by someone with the key

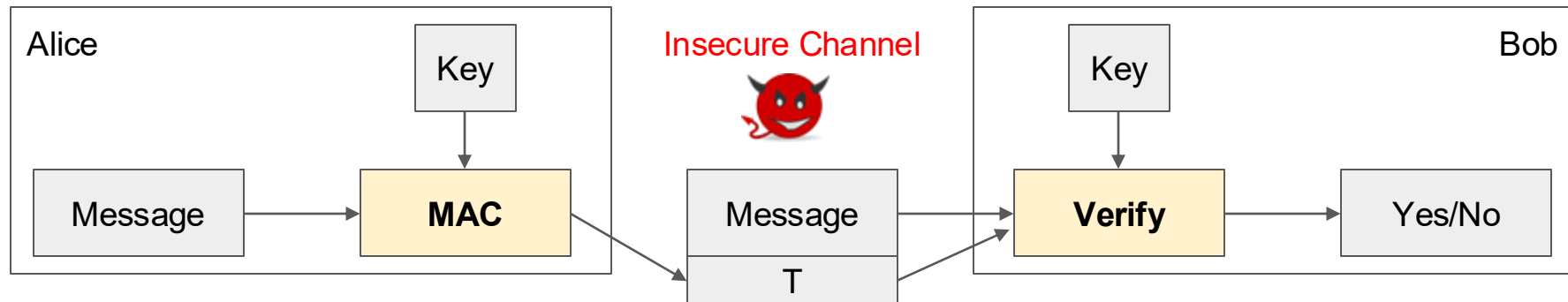
Message Authentication Codes (MACs)

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	● One-time pads ● Block ciphers (e.g. AES)	● RSA encryption ● ElGamal encryption
Integrity, Authentication	● Hash functions ● MACs (e.g. HMAC)	● Digital signatures (e.g. RSA signatures)

MACs: Usage

- Alice wants to send M to Bob, but doesn't want Mallory to tamper with it
- Alice sends M and $T = \text{MAC}(K, M)$ to Bob
- Bob recomputes $\text{MAC}(K, M)$ and checks that it matches T
- If the MACs match, Bob is confident the message has not been tampered with (integrity and authenticity)



MACs: Definition

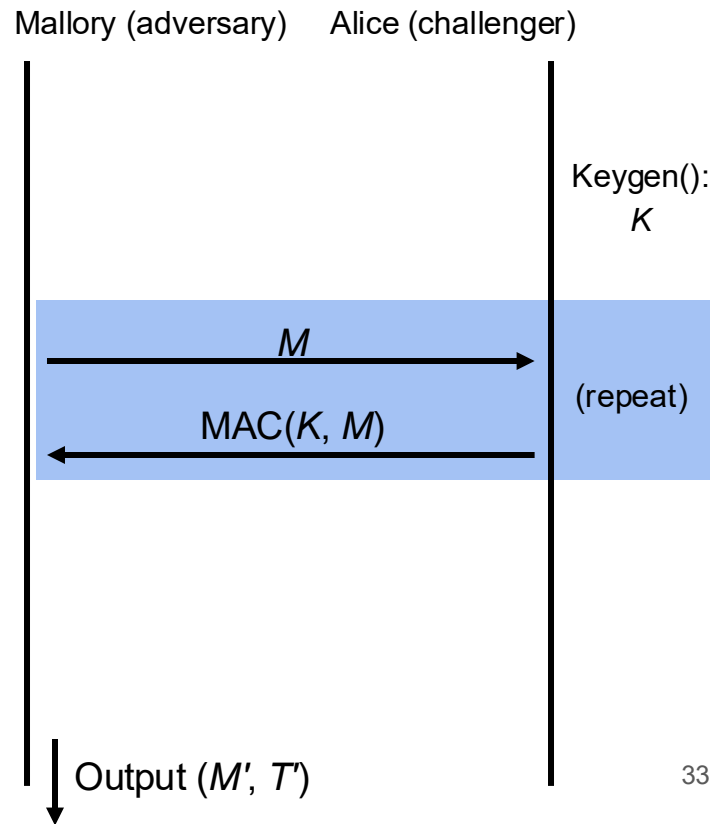
- Two parts:
 - $\text{KeyGen}() \rightarrow K$: Generate a key K
 - $\text{MAC}(K, M) \rightarrow T$: Generate a tag T for the message M using key K
 - Inputs: A secret key and an arbitrary-length message
 - Output: A fixed-length **tag** on the message
- Properties
 - **Correctness**: Determinism
 - **Efficiency**: Computing a MAC should be efficient
 - **Security**: EU-CPA (existentially unforgeable under chosen plaintext attack)

Defining Integrity: EU-CPA

- A secure MAC is **existentially unforgeable**: without the key, an attacker cannot create a valid tag on a message
 - Mallory cannot generate $\text{MAC}(K, M')$ without K
 - Mallory cannot find any $M' \neq M$ such that $\text{MAC}(K, M') = \text{MAC}(K, M)$
- Formally defined by a security game: existential unforgeability under chosen-plaintext attack, or EU-CPA
- MACs should be unforgeable under chosen plaintext attack
 - Intuition: Like IND-CPA, but for integrity and authenticity
 - Even if Mallory can trick Alice into creating MACs for messages that Mallory chooses, Mallory cannot create a valid MAC on a message that she hasn't seen before

Defining Integrity: EU-CPA

1. Mallory may send messages to Alice and receive their tags
2. Eventually, Mallory creates a message-tag pair (M', T')
 - M' cannot be a message that Mallory requested earlier
 - If T' is a valid tag for M' , then Mallory wins. Otherwise, she loses.
3. A scheme is EU-CPA secure if for *all* polynomial time adversaries, the probability of winning is 0 or negligible



Example: NMAC

- Can we use secure cryptographic hashes to build a secure MAC?
 - Intuition: Hash output is unpredictable and looks random, so let's hash the key and the message together
- KeyGen():
 - Output two random, n -bit keys K_1 and K_2 , where n is the length of the hash output
- NMAC(K_1, K_2, M):
 - Output $H(K_1 || H(K_2 || M))$
- NMAC is EU-CPA secure if the two keys are different
 - Provably secure if the underlying hash function is secure
- Intuition: Using two hashes prevents a length extension attack
 - Otherwise, an attacker who sees a tag for M could generate a tag for $M || M'$
- Issues with NMAC
 - We need two different keys
 - NMAC requires the keys to be the same length as the hash output (n bits)

Example: HMAC

- HMAC(K , M):
 - Compute K' as a version of K that is the length of the hash output
 - If K is too short, pad K with 0's to make it n bits (be careful with keys that are too short and lack randomness)
 - If K is too long, hash it so it's n bits
 - Output $H((K' \oplus opad) || H((K' \oplus ipad) || M))$
- Use K' to derive two different keys
 - *opad* (outer pad) is the hard-coded byte `0x5c` repeated until it's the same length as K'
 - *ipad* (inner pad) is the hard-coded byte `0x36` repeated until it's the same length as K'
 - As long as *opad* and *ipad* are different, you'll get two different keys
 - For paranoia, the designers chose two very different bit patterns, even though they theoretically need only differ in one bit

HMAC Properties

- $\text{HMAC}(K, M) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$
- HMAC is a hash function, so it has the properties of the underlying hash too
 - It is collision resistant
 - Given $\text{HMAC}(K, M)$ and K , an attacker can't learn M
 - If the underlying hash is secure, HMAC doesn't reveal M , but it is still deterministic
- You can't verify a tag T if you don't have K
 - The attacker can't brute-force the message M without knowing K

Do MACs provide integrity?

- Do MACs provide integrity?
 - Yes. An attacker cannot tamper with the message without being detected
- Do MACs provide authenticity?
 - It depends on your threat model
 - If a message has a valid MAC, you can be sure it came from *someone with the secret key*, but you can't narrow it down to one person
 - If only two people have the secret key, MACs provide authenticity: it has a valid MAC, and it's not from me, so it must be from the other person
- Do MACs provide confidentiality?
 - MACs are deterministic $\Rightarrow$ No IND-CPA security
 - MACs in general have no confidentiality guarantees; they can leak information about the message

MACs: Summary

- Inputs: a secret key and a message
- Output: a tag on the message
- A secure MAC is unforgeable: Even if Mallory can trick Alice into creating MACs for messages that Mallory chooses, Mallory cannot create a valid MAC on a message that she hasn't seen before
 - Example: $\text{HMAC}(K, M) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$
- MACs do not provide confidentiality

Authenticated Encryption: Definition

- **Authenticated encryption (AE):** A scheme that simultaneously guarantees confidentiality and integrity (and authenticity, depending on your threat model) on a message
- Two ways of achieving authenticated encryption:
 - Combine schemes that provide confidentiality with schemes that provide integrity
 - Use a scheme that is designed to provide confidentiality and integrity (not covered)

Combining Schemes: Let's design it together

- You can use:
 - An IND-CPA encryption scheme: $\text{Enc}(K, M)$ and $\text{Dec}(K, C)$
 - An unforgeable MAC scheme (e.g. HMAC): $\text{MAC}(K, M)$
- First attempt: Alice sends $\text{Enc}(K_1, M)$ and $\text{MAC}(K_2, M)$
 - Integrity? Yes, attacker can't tamper with the MAC
 - Confidentiality? No, the MAC is not IND-CPA secure
- Idea: Let's compute the MAC on the *ciphertext* instead of the plaintext:
 $\text{Enc}(K_1, M)$ and $\text{MAC}(K_2, \text{Enc}(K_1, M))$
 - Integrity? Yes, attacker can't tamper with the MAC
 - Confidentiality? Yes, the MAC might leak info about the ciphertext, but that's okay
- Idea: Let's encrypt the MAC too: $\text{Enc}(K_1, M \parallel \text{MAC}(K_2, M))$
 - Integrity? Yes, attacker can't tamper with the MAC
 - Confidentiality? Yes, everything is encrypted

MAC-then-Encrypt or Encrypt-then-MAC?

- MAC-then-encrypt
 - First compute $\text{MAC}(K_2, M)$
 - Then encrypt the message and the MAC together: $\text{Enc}(K_1, M \parallel \text{MAC}(K_2, M))$
- Encrypt-then-MAC
 - First compute $\text{Enc}(K_1, M)$
 - Then MAC the ciphertext: $\text{MAC}(K_2, \text{Enc}(K_1, M))$
- Which is better?
 - In theory, both are IND-CPA and EU-CPA secure if applied properly
 - MAC-then-encrypt has a downside: You don't know if tampering has occurred until after decrypting
 - Attacker can supply arbitrary tampered input, and you always have to decrypt it
 - Passing attacker-chosen input through the decryption function can cause side-channel leaks
- **Always use encrypt-then-MAC** because it's more robust to mistakes

Public-Key Encryption

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	● One-time pads ● Block ciphers (e.g. AES)	● RSA ● ElGamal encryption
Integrity, Authentication	● Hash functions ● MACs (e.g. HMAC)	● Digital signatures (e.g. RSA signatures)

Public-Key Encryption

- Everybody can encrypt with the public key
- Only the recipient can decrypt with the private key

Public-Key Encryption: Definition

- Three parts:
 - $\text{KeyGen}() \rightarrow PK, SK$: Generate a public/private keypair, where PK is the public key, and SK is the private (secret) key
 - $\text{Enc}(PK, M) \rightarrow C$: Encrypt a plaintext M using public key PK to produce ciphertext C
 - $\text{Dec}(SK, C) \rightarrow M$: Decrypt a ciphertext C using secret key SK
- Properties
 - **Correctness**: Decrypting a ciphertext should result in the message that was originally encrypted
 - $\text{Dec}(SK, \text{Enc}(PK, M)) = M$ for all $PK, SK \leftarrow \text{KeyGen}()$ and M
 - **Efficiency**: Encryption/decryption should be fast
 - **Security**: Similar to IND-CPA, but Alice (the challenger) just gives Eve (the adversary) the public key, and Eve doesn't request encryptions, except for the pair M_0, M_1
 - You don't need to worry about this game (it's called "semantic security")
- Example schemes: RSA, ElGamal
 - Security relies on hardness of factorization and discrete log problems

Digital Signatures

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	● One-time pads ● Block ciphers (e.g. AES)	● RSA encryption ● ElGamal encryption
Integrity, Authentication	● Hash functions ● MACs (e.g. HMAC)	● Digital signatures (e.g. RSA signatures)

Digital Signatures

- Asymmetric cryptography is good because we don't need to share a secret key
- Digital signatures are the asymmetric way of providing integrity/authenticity to data
- Assume that Alice and Bob can communicate public keys without Mallory changing them
 - We will see how to fix this limitation later using certificates
- Only the owner of the private key can sign messages with the private key
- Everybody can verify the signature with the public key

Digital Signatures: Definition

- Three parts:
 - $\text{KeyGen}() \rightarrow PK, SK$: Generate a public/private keypair, where PK is the verify (public) key, and SK is the signing (secret) key
 - $\text{Sign}(SK, M) \rightarrow sig$: Sign the message M using the signing key SK to produce the signature sig
 - $\text{Verify}(PK, M, sig) \rightarrow \{0, 1\}$: Verify the signature sig on message M using the verify key PK and output 1 if valid and 0 if invalid
- Properties
 - **Correctness**: Verification should be successful for a signature generated over any message
 - $\text{Verify}(PK, M, \text{Sign}(SK, M)) = 1$ for all $PK, SK \leftarrow \text{KeyGen}()$ and M
 - **Efficiency**: Signing/verifying should be fast
 - **Security**: EU-CPA, same as for MACs

Digital Signatures in Practice

- If you want to sign message M :
 - First hash M
 - Then sign $H(M)$
- Why do digital signatures use a hash?
 - Allows signing arbitrarily long messages
- Digital signatures provide integrity *and authenticity* for M
 - The digital signature acts as proof that the private key holder signed $H(M)$, so you know that M is authentically endorsed by the private key holder

Homework

- Check out the reading material for Lecture 2 and 3
- Homework exercise 1: Implement a function to find collisions